Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Архітектура комп'ютерів. Частина 2. Процесори

Лабораторна робота

№3

«Розгортання Linux Kernel в середовищі емулятора QEMU для архітектури ARM»

Виконав: студент
групи ІО-42
Куліков М. М.
Номер у списку групи: 9
Перевірив:
Каплунов А. В.

Київ – 2026

Лабораторна робота №3

Тема: «Розгортання Linux Kernel в середовищі емулятора QEMU для архітектури ARM».

Мета: Зібрати програмне середовище, завантажувач U-Boot, ядро Linux та BusyBox для архітектури ARM та запустити систему в QEMU.

Хід роботи:

Роботу виконано на операційній системі Ubuntu 22.04 LTS. Для початку перевіримо версію системи та компілятора gcc командами lsb_release -a та gcc --version. Результат наведено нижче.

```
maksym@acer:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.5 LTS
Release:        22.04
Codename:       jammy
maksym@acer:~$ gcc --version
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Рисунок 1 – Версії операційної системи та gcc

Оскільки в системі за замовчуванням встановлена новіша версія компілятора (що може призводити до помилок на кшталт yylloc під час збірки старого коду завантажувача), встановимо сумісну 9-ту версію GCC та зареєструємо її в системі за допомогою наступних команд:

```
sudo apt update
sudo apt install gcc-9 g++-9
sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-9 9
gcc --version
```

```
Preparing to unpack .../libasan5_9.5.0-1ubuntu1~22.04.1_amd64.deb ...
Unpacking libasan5:amd64 (9.5.0-1ubuntu1~22.04.1) ...
Selecting previously unselected package libgcc-9-dev:amd64.
Preparing to unpack .../libgcc-9-dev_9.5.0-1ubuntu1~22.04.1_amd64.deb ...
Unpacking libgcc-9-dev:amd64 (9.5.0-1ubuntu1~22.04.1) ...
Selecting previously unselected package gcc-9.
Preparing to unpack .../gcc-9_9.5.0-1ubuntu1~22.04.1_amd64.deb ...
Unpacking gcc-9 (9.5.0-1ubuntu1~22.04.1) ...
Setting up gcc-9-base:amd64 (9.5.0-1ubuntu1~22.04.1) ...
Setting up libasan5:amd64 (9.5.0-1ubuntu1~22.04.1) ...
Setting up cpp-9 (9.5.0-1ubuntu1~22.04.1) ...
Setting up libgcc-9-dev:amd64 (9.5.0-1ubuntu1~22.04.1) ...
Setting up gcc-9 (9.5.0-1ubuntu1~22.04.1) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.15) ...
maksym@acer:~$ sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-
9 9
update-alternatives: using /usr/bin/gcc-9 to provide /usr/bin/gcc (gcc) in auto
mode
maksym@acer:~$ gcc --version
gcc (Ubuntu 9.5.0-1ubuntu1~22.04.1) 9.5.0
Copyright (C) 2019 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Рисунок 2 – Зміна gcc

Частина 1. Підготовка інструментів

Оновимо список пакетів і встановимо інструменти, які знадобляться надалі: git для отримання вихідного коду, vim як редактор, tree та curl.

```
sudo apt update
sudo apt install git vim vim-gtk tree curl
```

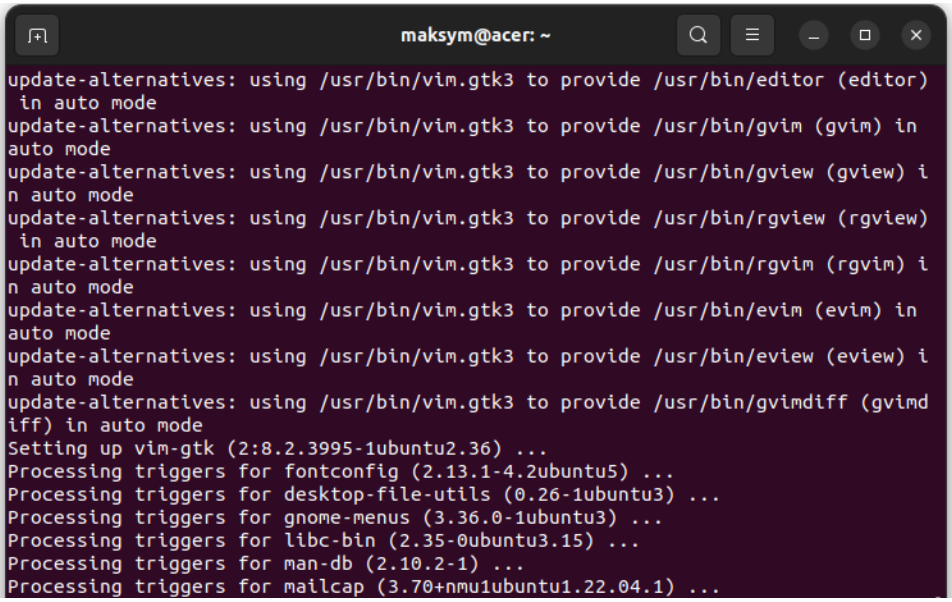


Рисунок 3 – Оновлення пакетів і встановлення базових інструментів

Далі встановимо ccache, який кешує об’єктні файли й прискорює повторні збірки, зокрема ядра. Встановимо розмір кешу 5 ГБ та перевіримо статистику.

```
sudo apt install ccache
ccache -M 5G
ccache -s
```

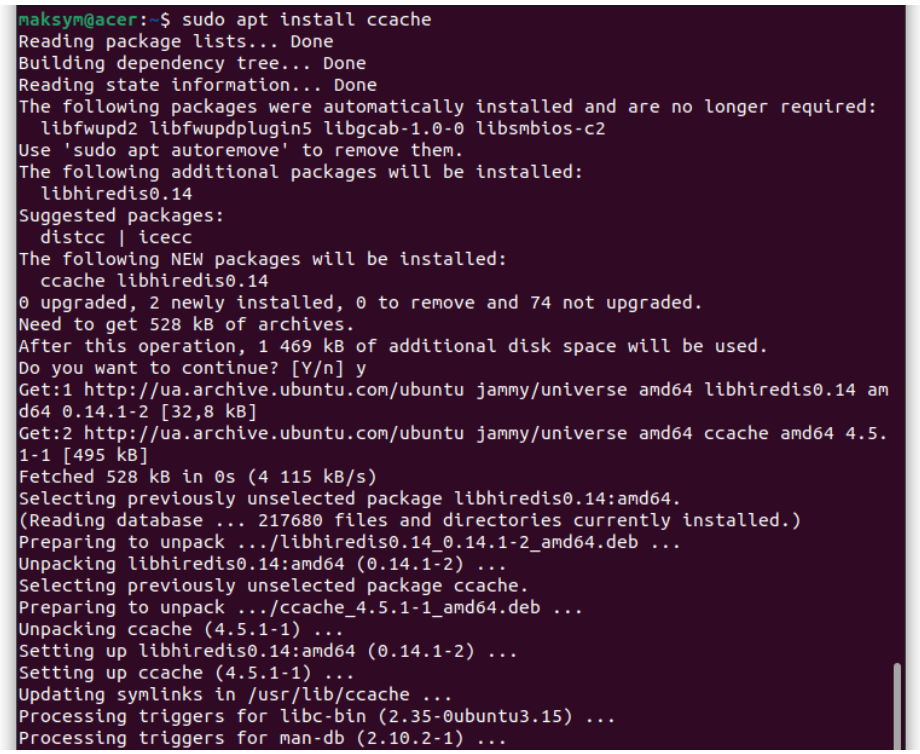


Рисунок 4 – Встановлення ccache

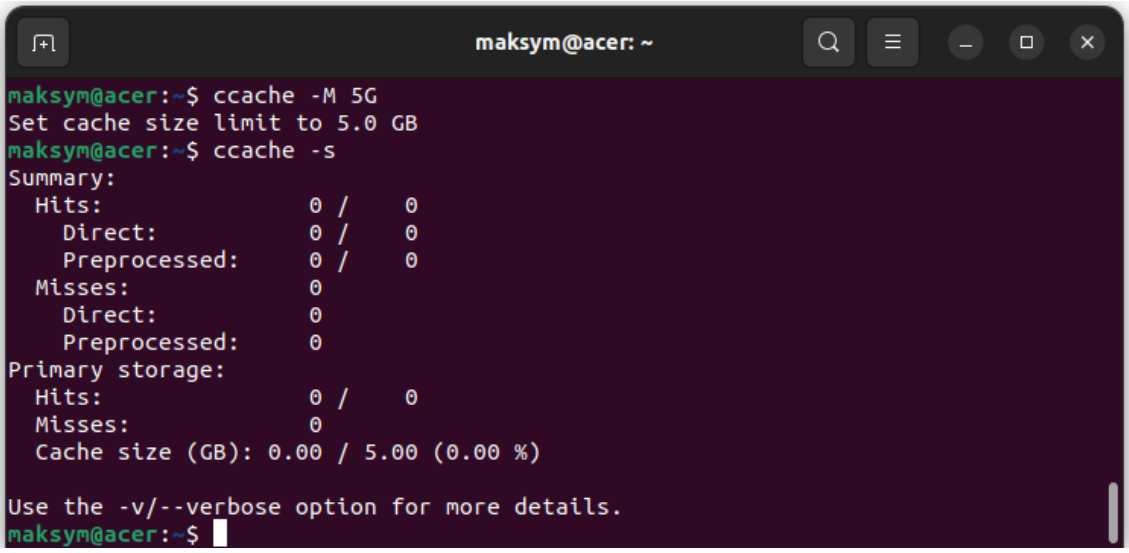


Рисунок 5 – Налаштування розміру та статистика ccache

Оскільки замість фізичної плати використовується емулятор QEMU, пункти 1.3 (послідовна консоль) і 1.4 (udev-правила для OTG USB) пропускаємо. Переходимо до тулчейнів. Нам потрібні два: bare-metal (arm-eabi) для збірки U-Boot і ядра та Linux (arm-linux-gnueabihf) для збірки BusyBox. Завантажимо архіви зі сайту ARM у каталог ~/Downloads.

```
wget https://developer.arm.com/-/media/Files/downloads/gnu-a/8.3-2019.03/binrel/gcc-arm-8.3-2019.03-x86_64-arm-eabi.tar.xz
wget https://developer.arm.com/-/media/Files/downloads/gnu-a/8.3-2019.03/binrel/gcc-arm-8.3-2019.03-x86_64-arm-linux-gnueabihf.tar.xz
```

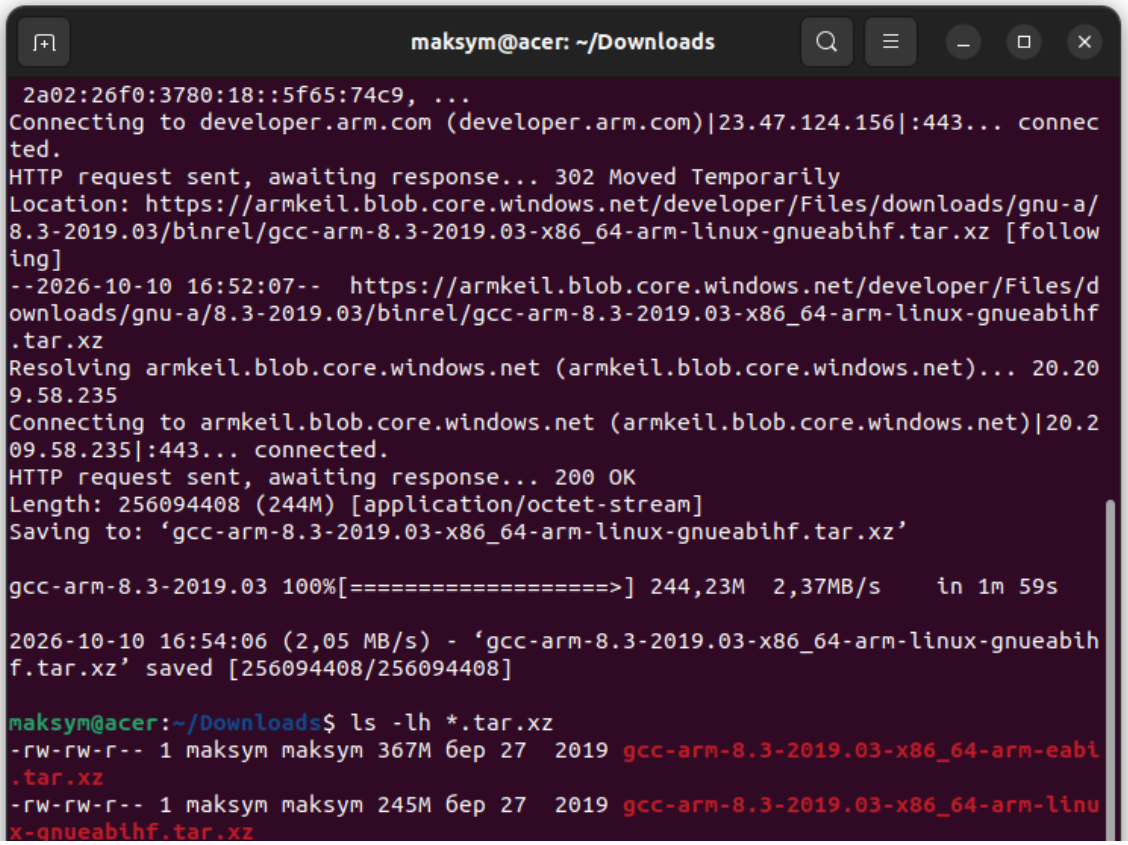


Рисунок 6 – Завантажені архіви тулчейнів

Розпакуємо обидва архіви в каталог /opt.

```
sudo tar xJvf gcc-arm-8.3-2019.03-x86_64-arm-eabi.tar.xz -C /opt/
sudo tar xJvf gcc-arm-8.3-2019.03-x86_64-arm-linux-gnueabihf.tar.xz -C /opt/
```

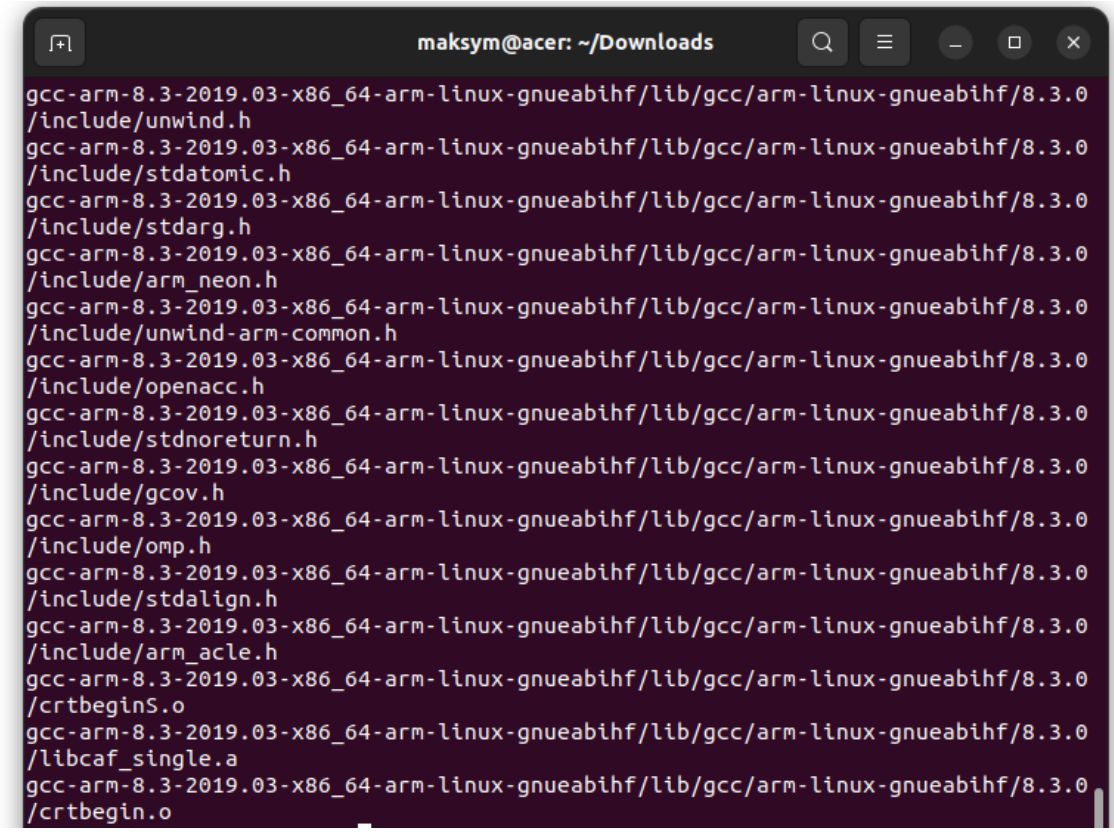


Рисунок 7 – Розпакування тулчейнів до каталогу /opt

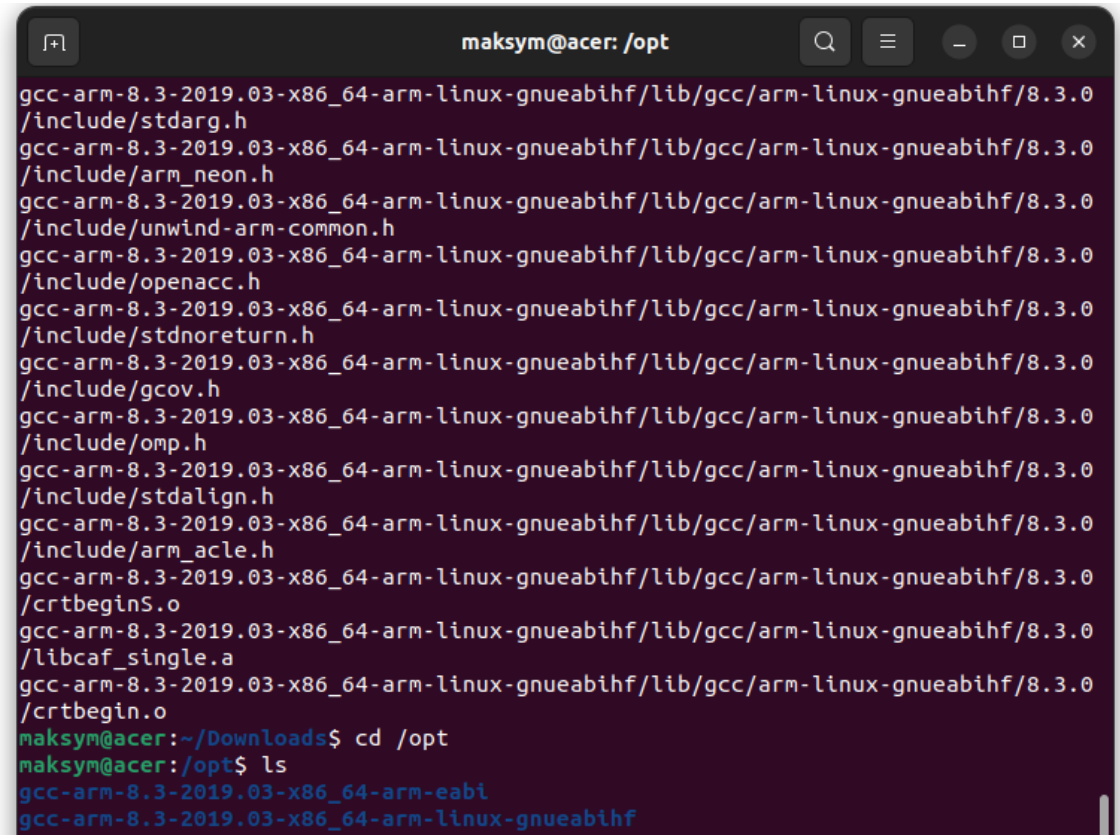


Рисунок 8 – Вміст каталогу /opt

Частина 2. Отримання та збірка програмного забезпечення

Встановимо залежності, потрібні для збірки всіх проєктів, і створимо каталог ~/repos для вихідного коду.

```
sudo apt install make libncurses5-dev libssl-dev bc bison flex
mkdir ~/repos
```

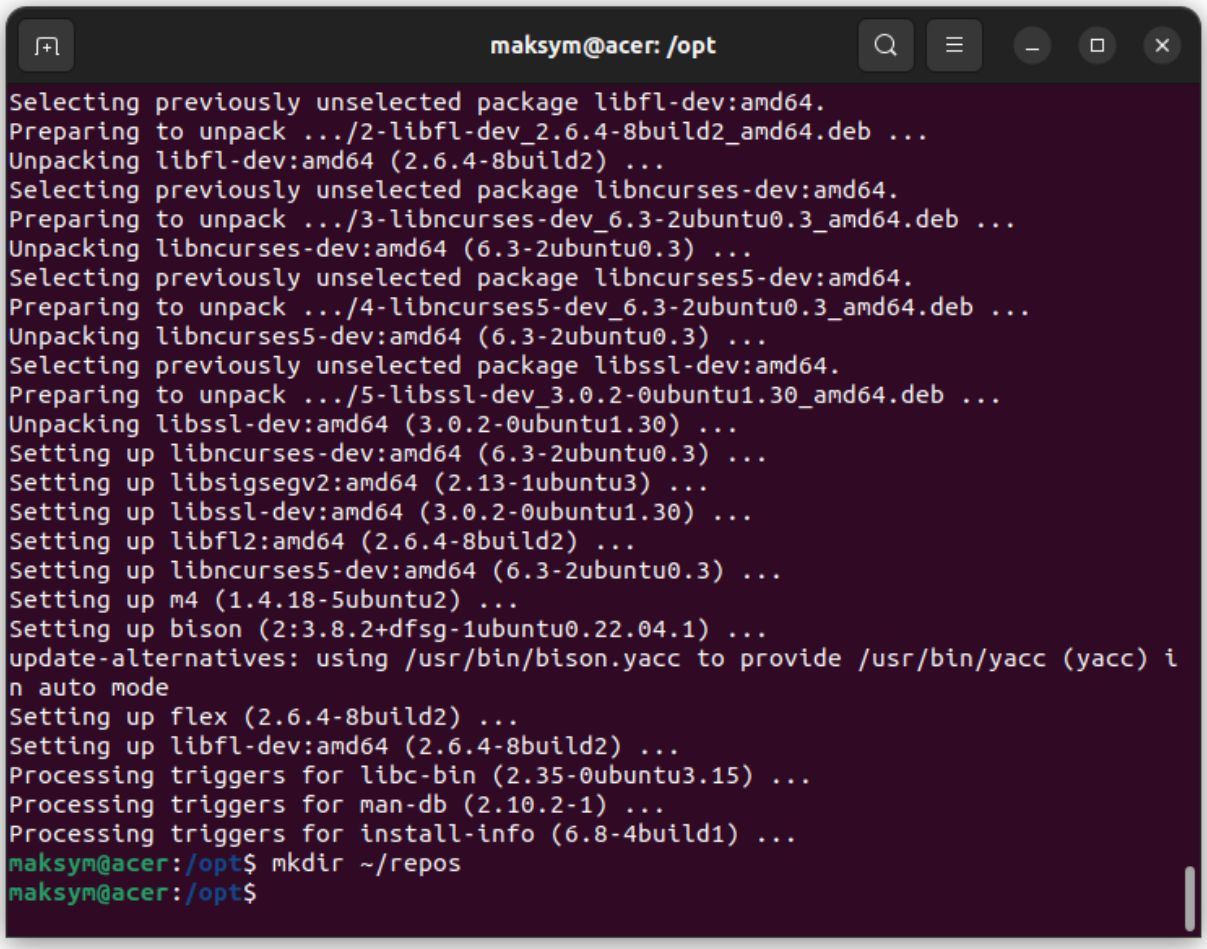


Рисунок 9 – Встановлення залежностей для збірки

2.1 U-Boot

U-Boot — це завантажувач, який широко використовується у вбудованих системах. Клонуємо репозиторій, переглянемо останні релізні теги (без rc) і перейдемо на тег v2019.07.

```
cd ~/repos
git clone https://gitlab.denx.de/u-boot/u-boot.git
cd u-boot
git tag | grep -v rc | tail -5
git checkout v2019.07
```

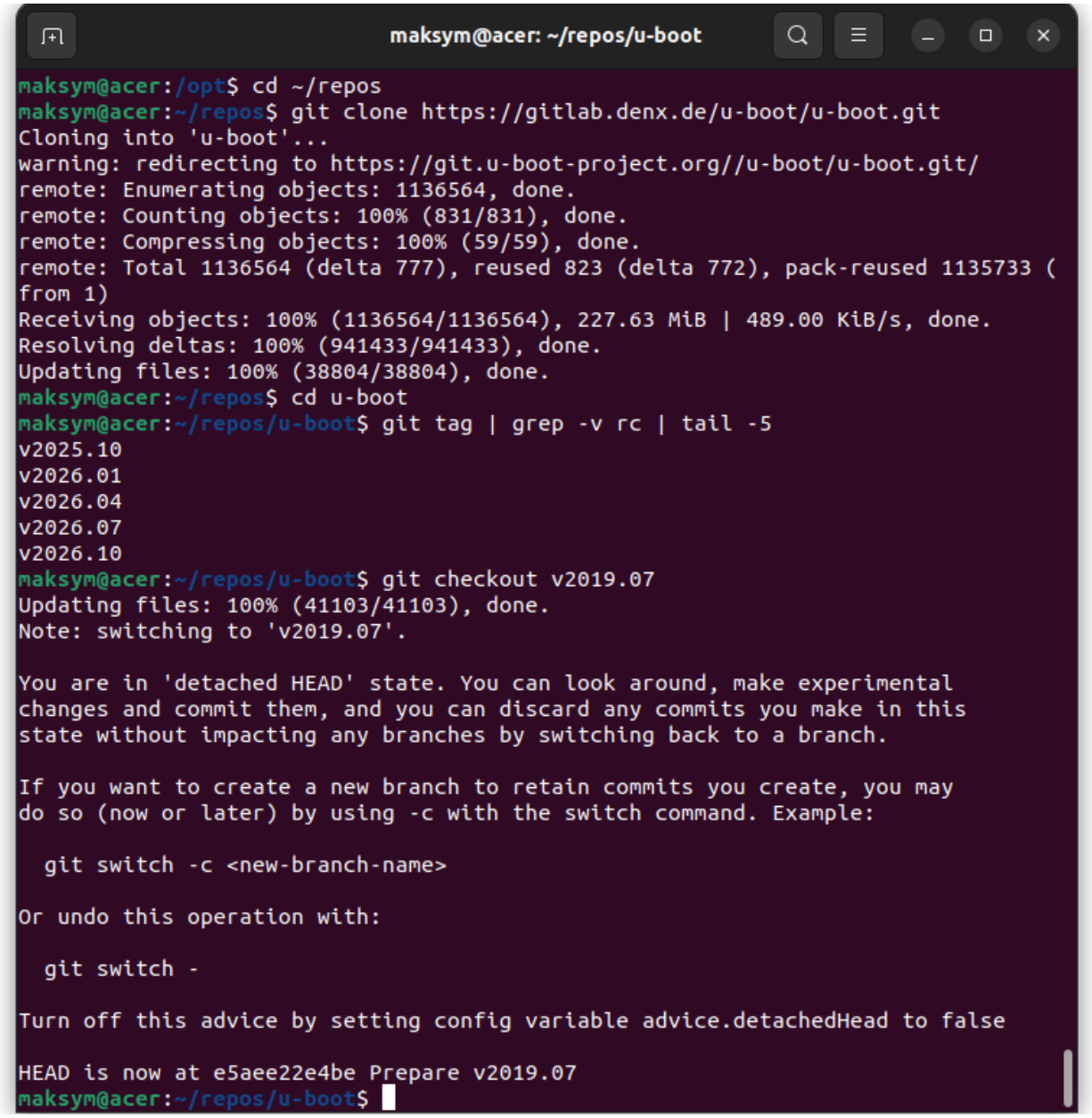


Рисунок 10 – Отримання вихідного коду U-Boot

Застосуємо серію патчів, яка ще очікує рецензування.

```
curl https://patchwork.ozlabs.org/series/130450/mbox/ | git am
```

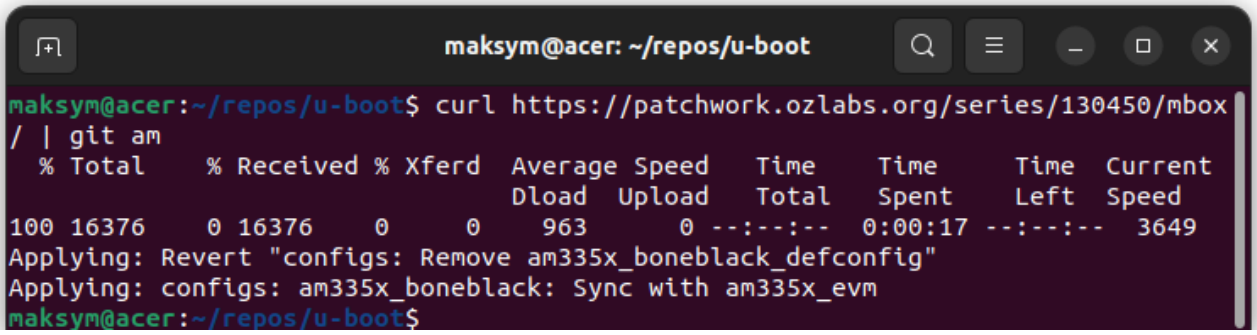


Рисунок 11 – Застосування патч-серії до U-Boot

Налаштуємо середовище для крос-компіляції bare-metal тулчейном. Ці команди не додаємо до ~/.bashrc.

```
export PATH=/opt/gcc-arm-8.3-2019.03-x86_64-arm-eabi/bin:$PATH
export CROSS_COMPILE='ccache arm-eabi-'
export ARCH=arm
```

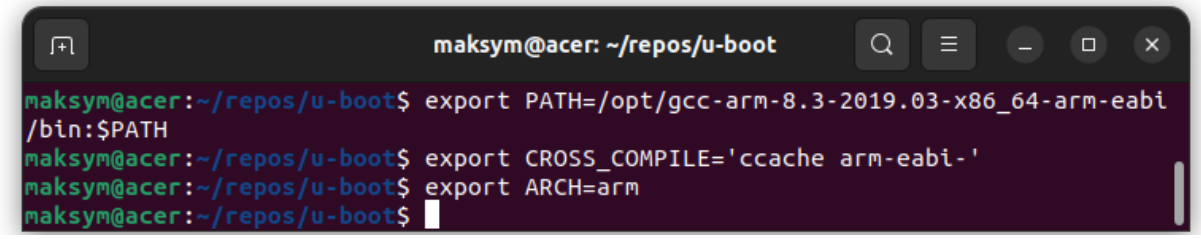


Рисунок 12 – Налаштування середовища для збірки U-Boot

Зберемо U-Boot з конфігурацією для BeagleBone Black.

```
make am335x_boneblack_defconfig
make -j4
```

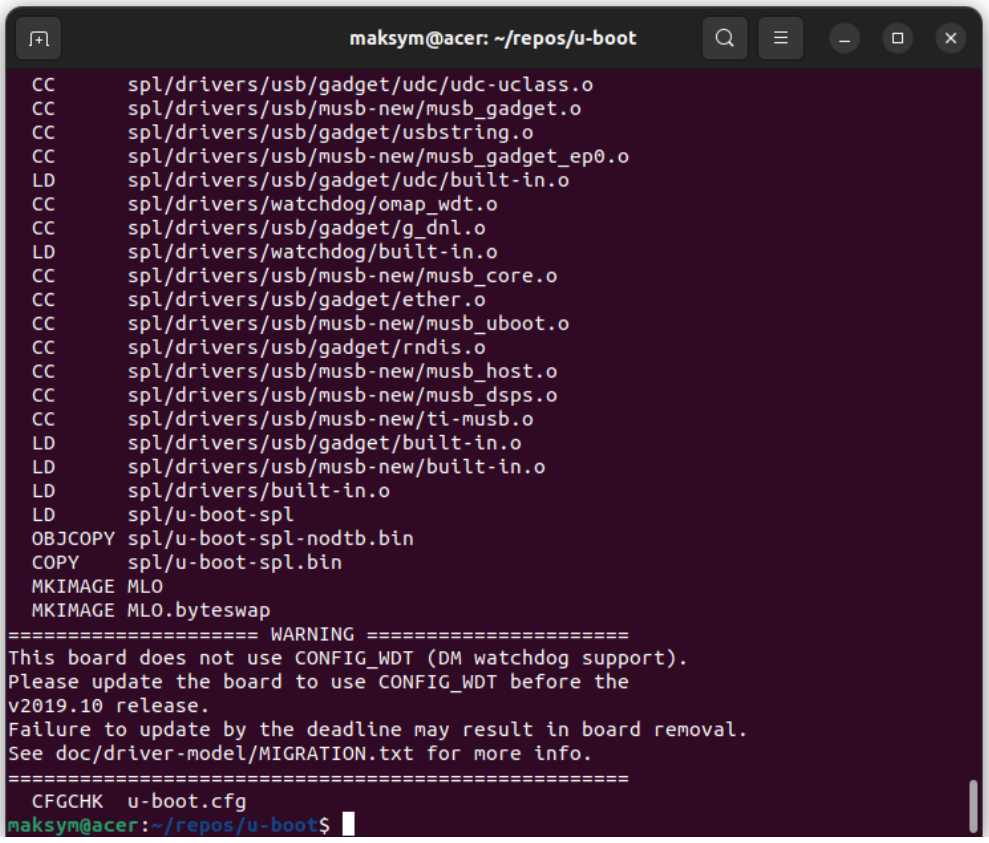


Рисунок 13 – Збірка U-Boot

Результатом збірки є файли MLO (завантажувач першого етапу) та u-boot.img (завантажувач другого етапу).

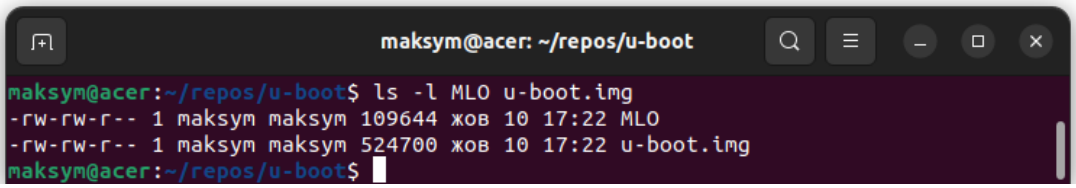


Рисунок 14 – Результат збірки U-Boot

2.2 Ядро Linux

Клонуємо репозиторій linux-stable і перейдемо на останню longterm-гілку linux-4.19.y.

```
cd ~/repos
git clone git://git.kernel.org/pub/scm/linux/kernel/git/stable/linux-stable.git
cd linux-stable
git checkout linux-4.19.y
```

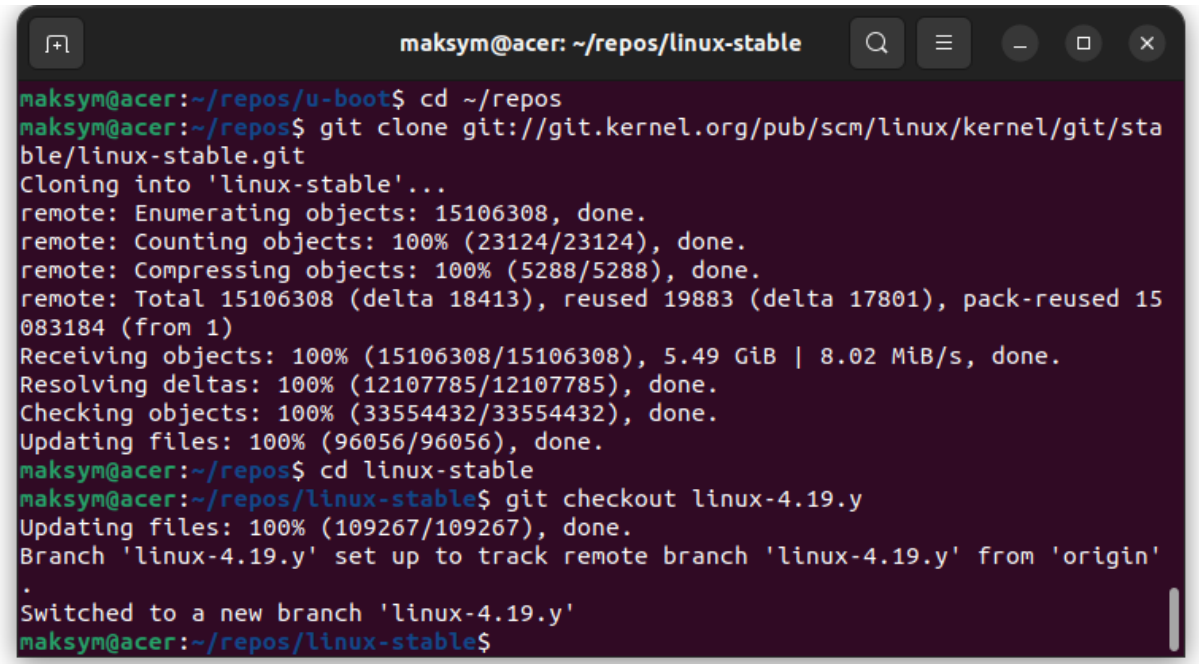


Рисунок 15 – Отримання вихідного коду ядра

Налаштуємо середовище, як і для U-Boot. За основу конфігурації візьмемо multi_v7_defconfig, а потрібні для BBB опції додамо фрагментом fragments/bbb.cfg.

```
export PATH=/opt/gcc-arm-8.3-2019.03-x86_64-arm-eabi/bin:$PATH
export CROSS_COMPILE='ccache arm-eabi-'
export ARCH=arm
mkdir fragments
nano fragments/bbb.cfg
```

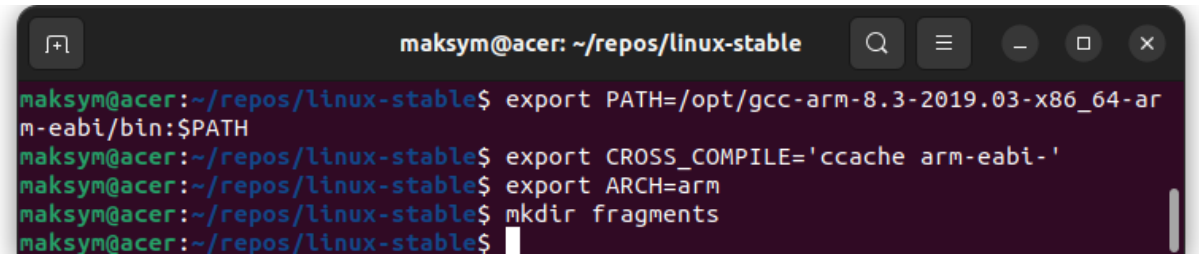


Рисунок 16 – Налаштування середовища для збірки ядра

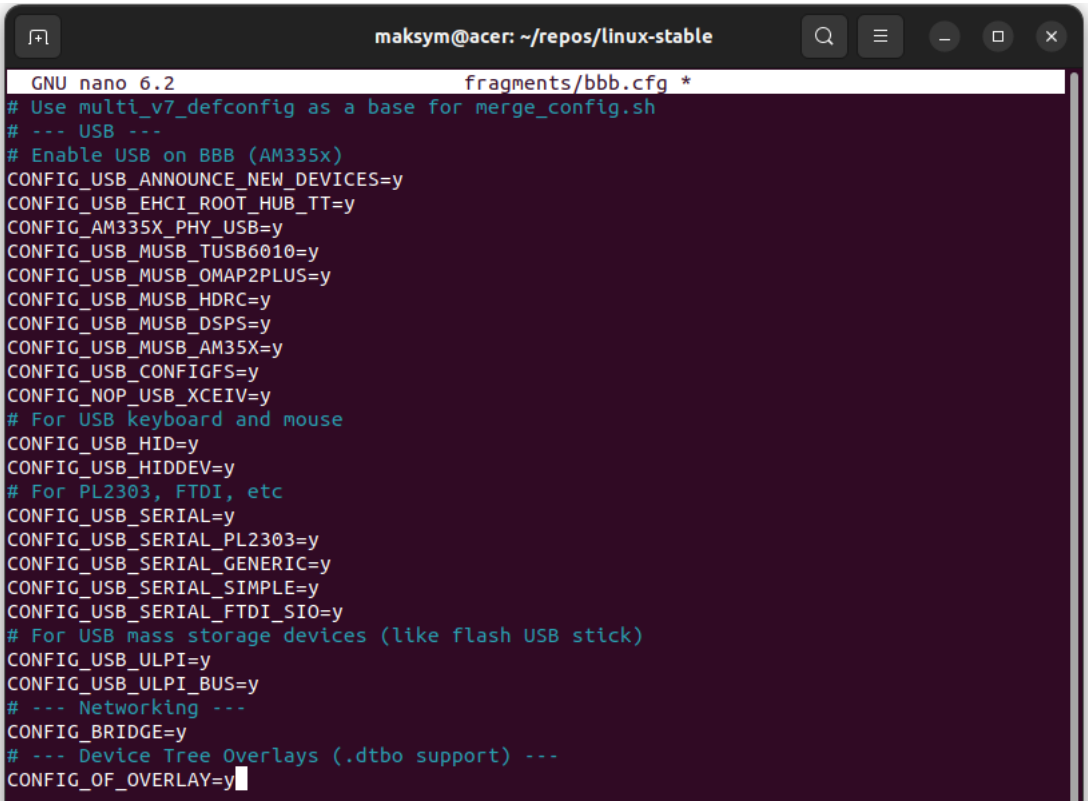


Рисунок 17 – Вміст файлу bbb.cfg

Згенеруємо файл .config з multi_v7_defconfig та нашого фрагмента.

```
./scripts/kconfig/merge_config.sh \
  arch/arm/configs/multi_v7_defconfig fragments/bbb.cfg
```

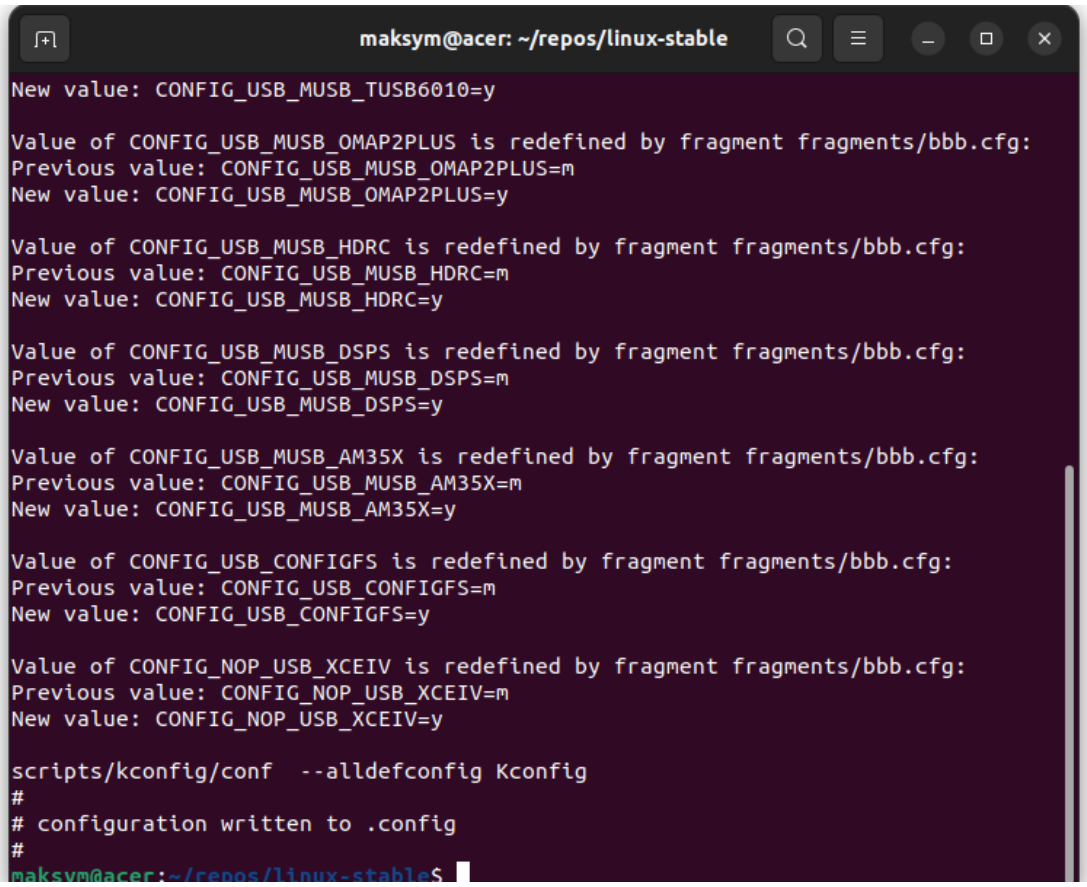


Рисунок 18 – Генерація .config

Зберемо образ ядра, device tree і модулі.

```
make -j4 zImage modules am335x-boneblack.dtb
```

```
  LD [M]  sound/soc/samsung/snd-soc-i2s.ko
  LD [M]  sound/soc/samsung/snd-soc-odroid.ko
  LD [M]  sound/soc/samsung/snd-soc-pcm.ko
  LD [M]  sound/soc/samsung/snd-soc-s3c-dma.ko
  LD [M]  sound/soc/samsung/snd-soc-smdk-wm8994.ko
  LD [M]  sound/soc/samsung/snd-soc-smdk-wm8994pcm.ko
  LD [M]  sound/soc/samsung/snd-soc-snow.ko
  LD [M]  sound/soc/sh/rcar/snd-soc-rcar.ko
  LD [M]  sound/soc/sh/snd-soc-fsi.ko
  LD [M]  sound/soc/snd-soc-core.ko
  LD [M]  sound/soc/sti/snd-soc-sti.ko
  LD [M]  sound/soc/sunxi/sun4i-codec.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra-alc5632.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra-max98090.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra-pcm.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra-rt5640.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra-trimslice.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra-utils.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra-wm8753.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra-wm8903.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra-wm9712.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra20-ac97.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra20-das.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra20-i2s.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra20-spdif.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra30-ahub.ko
  LD [M]  sound/soc/tegra/snd-soc-tegra30-i2s.ko
  LD [M]  sound/soundcore.ko
  LD [M]  sound/usb/snd-usb-audio.ko
  LD [M]  sound/usb/snd-usbmidi-lib.ko
maksym@acer:~/repos/linux-stable$
```

Рисунок 19 – Збірка ядра Linux

Результатом є файли arch/arm/boot/zImage та arch/arm/boot/dts/am335x-boneblack.dtb.

```
maksym@acer: ~/repos/linux-stable
maksym@acer:~/repos/linux-stable$ ls -lh arch/arm/boot/zImage arch/arm/boot/dts/am335x-boneblack.dtb
-rw-rw-r-- 1 maksym maksym  35K жов 10 18:25 arch/arm/boot/dts/am335x-boneblack.dtb
-rwxrwxr-x 1 maksym maksym 8,6M жов 10 18:36 arch/arm/boot/zImage
maksym@acer:~/repos/linux-stable$
```

Рисунок 20 – Результат збірки ядра

2.3 BusyBox

Зберемо мінімальну кореневу файлову систему на основі BusyBox. Клонуємо репозиторій і перейдемо на гілку 1_31_stable.

```
cd ~/repos
git clone git://git.busybox.net/busybox
cd busybox
git branch -a | grep stable | sort -V | tail -1
git checkout 1_31_stable
```

```
maksym@acer: ~/repos/busybox
maksym@acer:~/repos/linux-stable$ cd ~/repos
maksym@acer:~/repos$ git clone git://git.busybox.net/busybox
Cloning into 'busybox'...
remote: Enumerating objects: 1398, done.
remote: Counting objects: 100% (1398/1398), done.
remote: Compressing objects: 100% (1395/1395), done.
remote: Total 120948 (delta 966), reused 0 (delta 0), pack-reused 119550
Receiving objects: 100% (120948/120948), 30.97 MiB | 1.89 MiB/s, done.
Resolving deltas: 100% (96898/96898), done.
maksym@acer:~/repos$ cd busybox
maksym@acer:~/repos/busybox$ git branch -a | grep stable | sort -V | tail -1
  remotes/origin/1_37_stable
maksym@acer:~/repos/busybox$ git checkout 1_31_stable
Branch '1_31_stable' set up to track remote branch '1_31_stable' from 'origin'.
Switched to a new branch '1_31_stable'
maksym@acer:~/repos/busybox$
```

Рисунок 21 – Отримання вихідного коду BusyBox

Тепер використовуємо Linux-тулчейн. Налаштуємо середовище та згенеруємо повну конфігурацію.

```
export ARCH=arm
export PATH=/opt/gcc-arm-8.3-2019.03-x86_64-arm-linux-gnueabihf/bin:$PATH
export CROSS_COMPILE="ccache arm-linux-gnueabihf-"
make defconfig
```

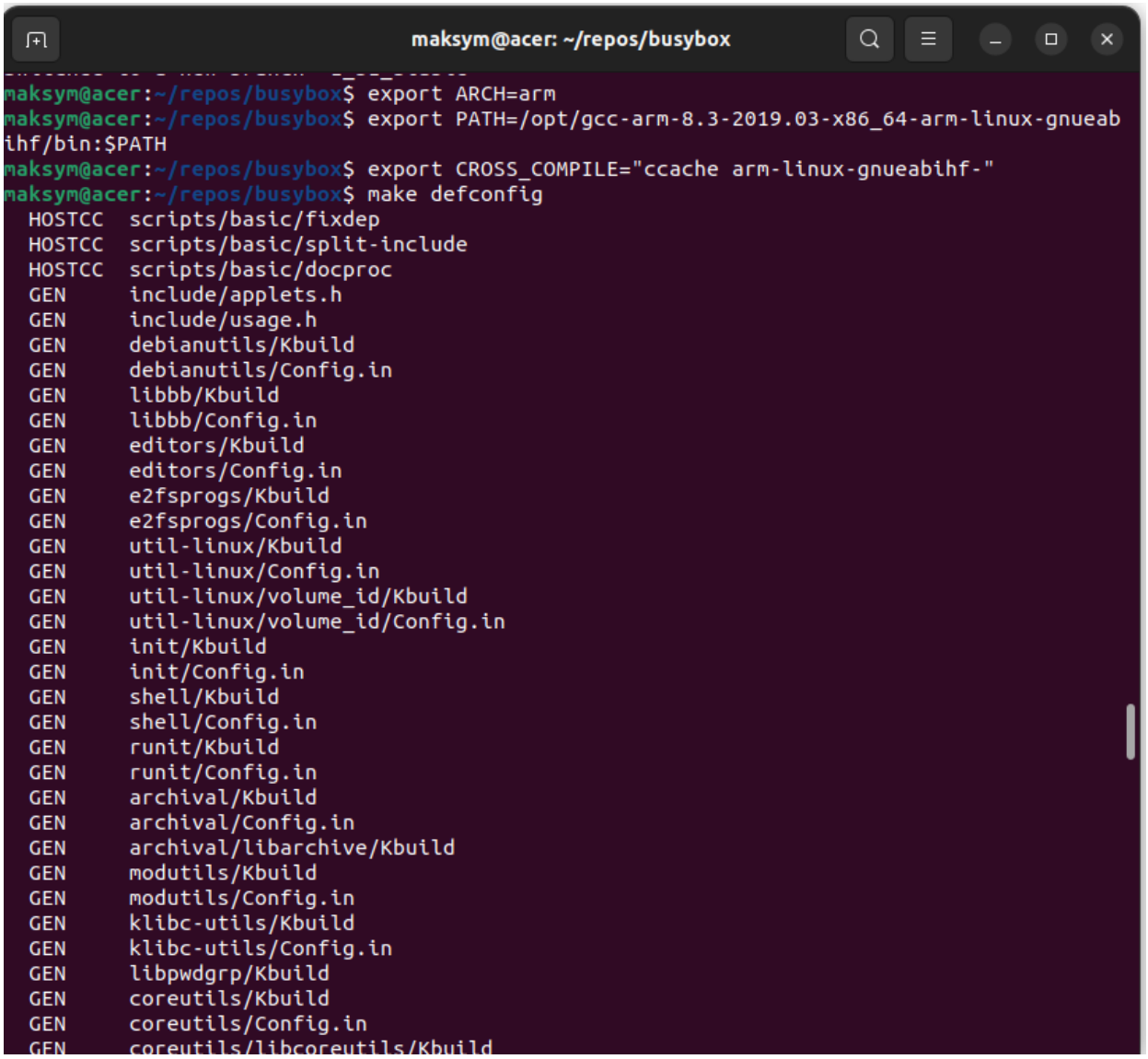


Рисунок 22 – Налаштування середовища та конфігурації BusyBox

Зберемо BusyBox, встановимо його в каталог _install та створимо потрібні каталоги.

```
make -j4
make install
mkdir -p
_install/{boot,dev,etc\/init.d,lib,proc,root,sys\/kernel\/debug,tmp}
```

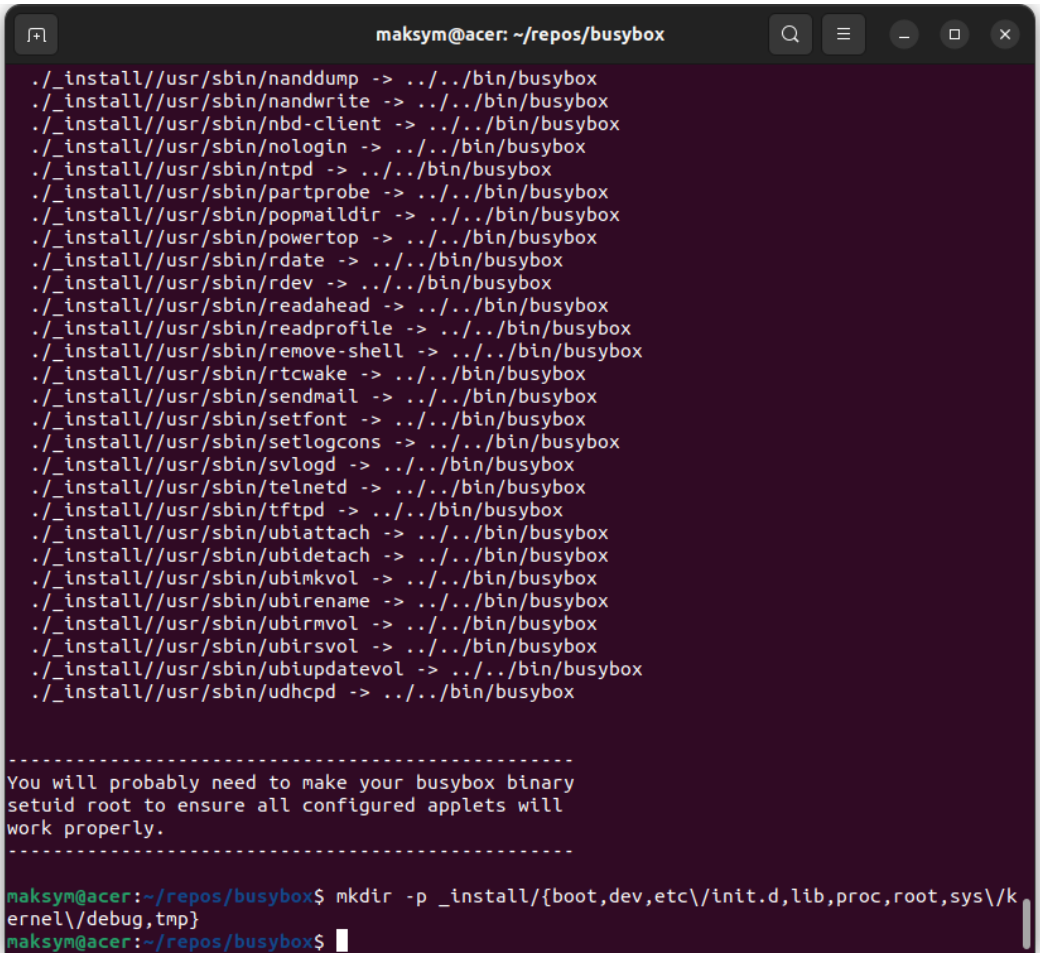


Рисунок 23 – Збірка та встановлення BusyBox

Створимо скрипт ініціалізації _install/etc/init.d/rcS, зробимо його виконуваним і створимо посилання init, щоб ядро могло запустити систему.

```
#!/bin/sh

mount -t sysfs none /sys
mount -t proc none /proc
mount -t debugfs none /sys/kernel/debug

echo /sbin/mdev > /proc/sys/kernel/hotplug
mdev -s
```

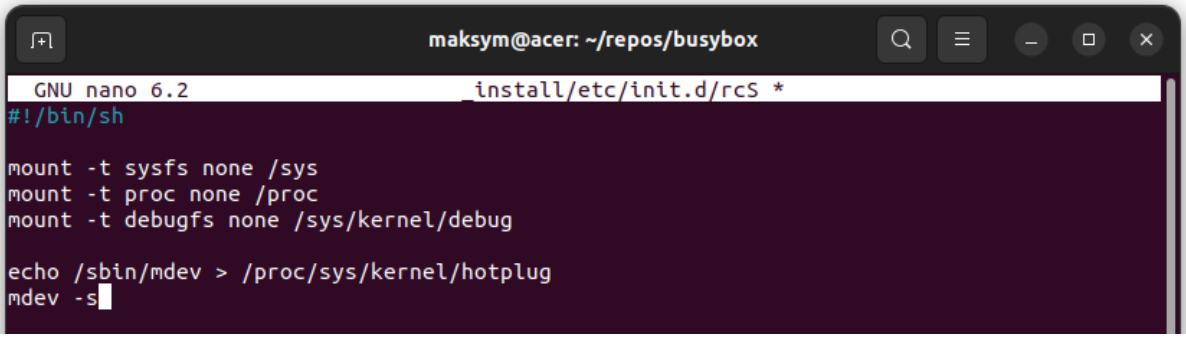


Рисунок 24 – Скрипт ініціалізації rcS

```
chmod +x _install/etc/init.d/rcS
ln -s bin/busybox _install/init
```

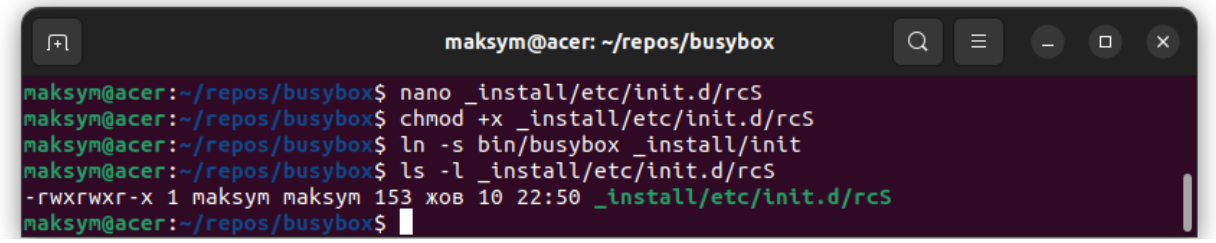


Рисунок 25 – Права доступу до rcS та посилання init

Наповнимо каталог /boot файлами ядра: zImage, am335x-boneblack.dtb, System.map та .config.

```
cd _install/boot
cp ~/repos/linux-stable/arch/arm/boot/zImage .
cp ~/repos/linux-stable/arch/arm/boot/dts/am335x-boneblack.dtb .
cp ~/repos/linux-stable/System.map .
cp ~/repos/linux-stable/.config ./config
cd -
```

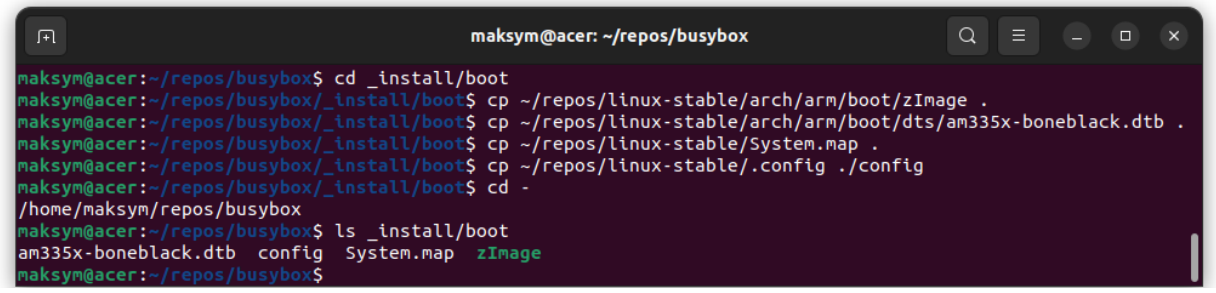


Рисунок 26 – Наповнення каталогу /boot

Наповнимо каталог /lib. Спочатку встановимо модулі ядра в кореневу файлову систему.

```
cd ~/repos/linux-stable
export INSTALL_MOD_PATH=~/repos/busybox/_install
export ARCH=arm
make modules_install
cd -
```

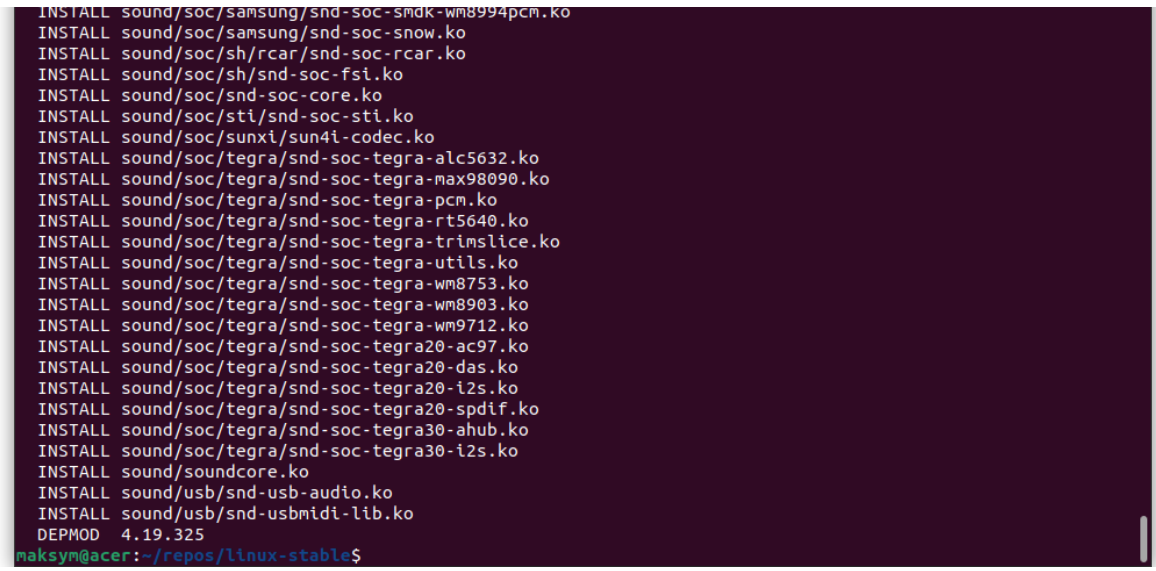


Рисунок 27 – Встановлення модулів ядра

Оскільки BusyBox зібрано динамічно, потрібно скопіювати бібліотеки тулчейна. Перевіримо залежності бінарного файлу та скопіюємо бібліотеки.

```
${CROSS_COMPILE}readelf -d _install/bin/busybox | grep NEEDED
cd _install/lib
libc_dir=$(${CROSS_COMPILE}gcc -print-sysroot)/lib
cp -a $libc_dir/*.so* .
cd -
```

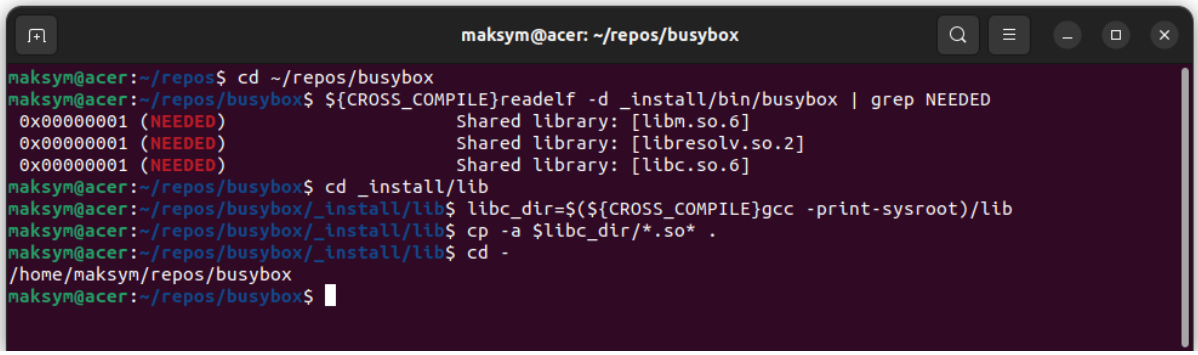


Рисунок 28 – Копіювання системних бібліотек у /lib

Наповнимо каталог /etc: конфігурація mdev, файли group, passwd, shadow і resolv.conf.

```
echo '$MODALIAS=.* root:root 660 @modprobe "$MODALIAS"' >
_install/etc/mdev.conf
echo 'root:x:0:' > _install/etc/group
echo 'root:x:0:0:root:/root:/bin/sh' > _install/etc/passwd
echo 'root::10933:0:99999:7:::' > _install/etc/shadow
echo "nameserver 8.8.8.8" > _install/etc/resolv.conf
```

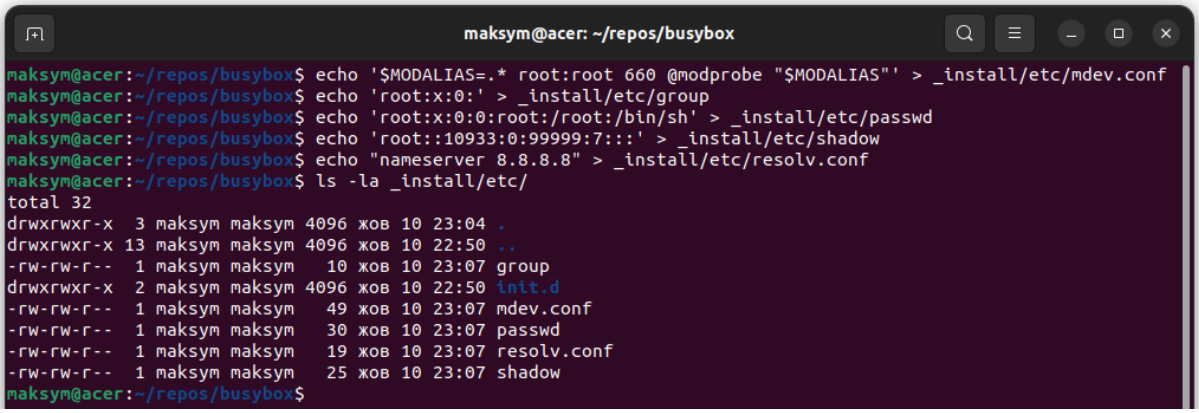


Рисунок 29 – Наповнення каталогу /etc

Частина 3. Запуск в емуляторі QEMU

Встановимо QEMU та створимо стиснений CPIO-архів кореневої файлової системи.

```
sudo apt install qemu-system-arm
cd ~/repos/busybox/_install
find . | cpio -o -H newc | gzip > ../rootfs.cpio.gz
cd ..
```

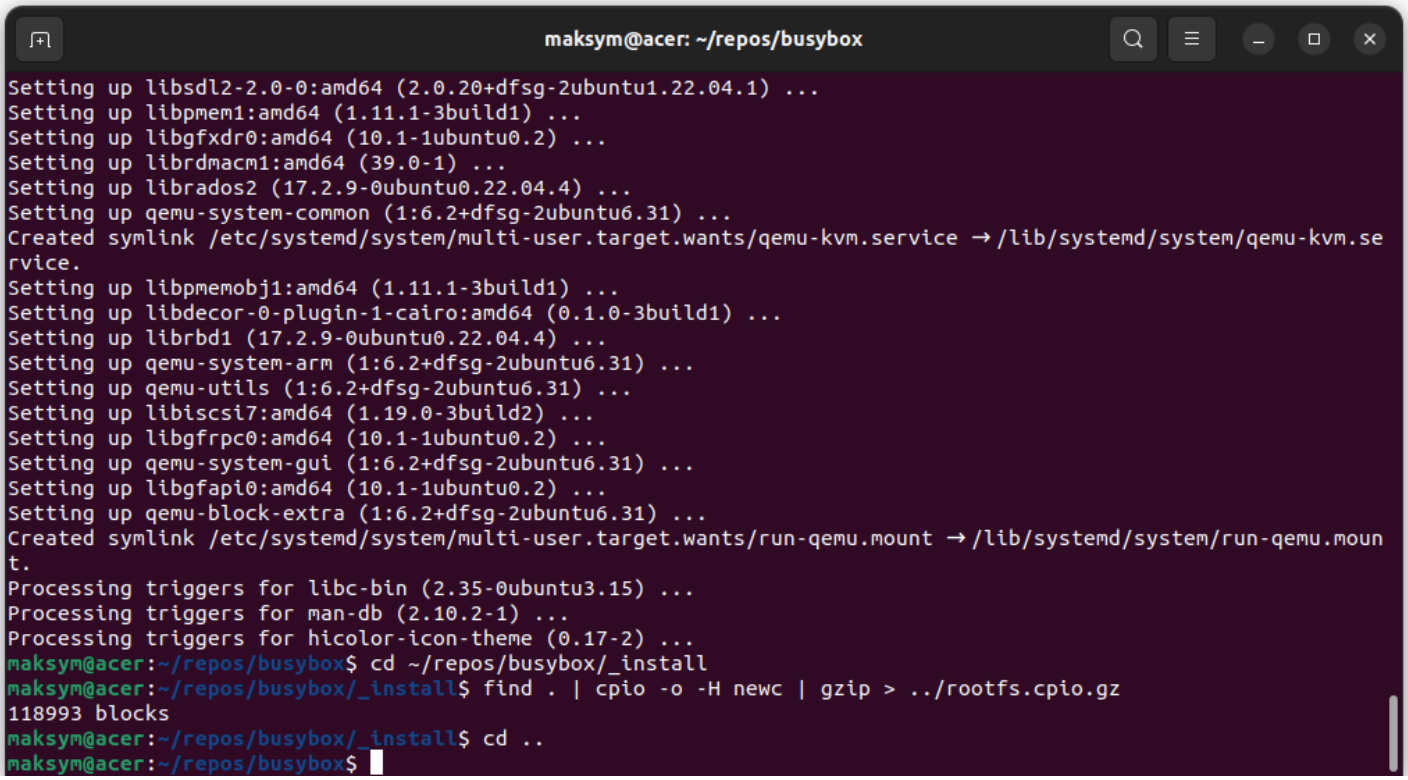


Рисунок 30 – Встановлення QEMU

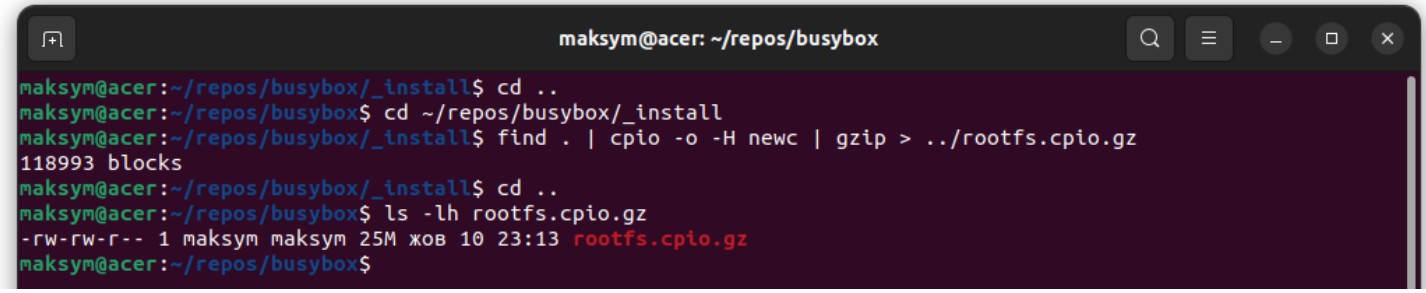


Рисунок 31 – Створення CPIO-архіву кореневої файлової системи

Запустимо ядро з кореневою файловою системою в QEMU. Використовується машина virt, тому dtb і завантажувач не потрібні, а rootfs підключається як RAM-диск.

```
qemu-system-arm -kernel _install/boot/zImage -initrd rootfs.cpio.gz \
  -machine virt -nographic -m 512 \
  --append "root=/dev/ram0 rw console=ttyAMA0,115200 mem=512M"
```

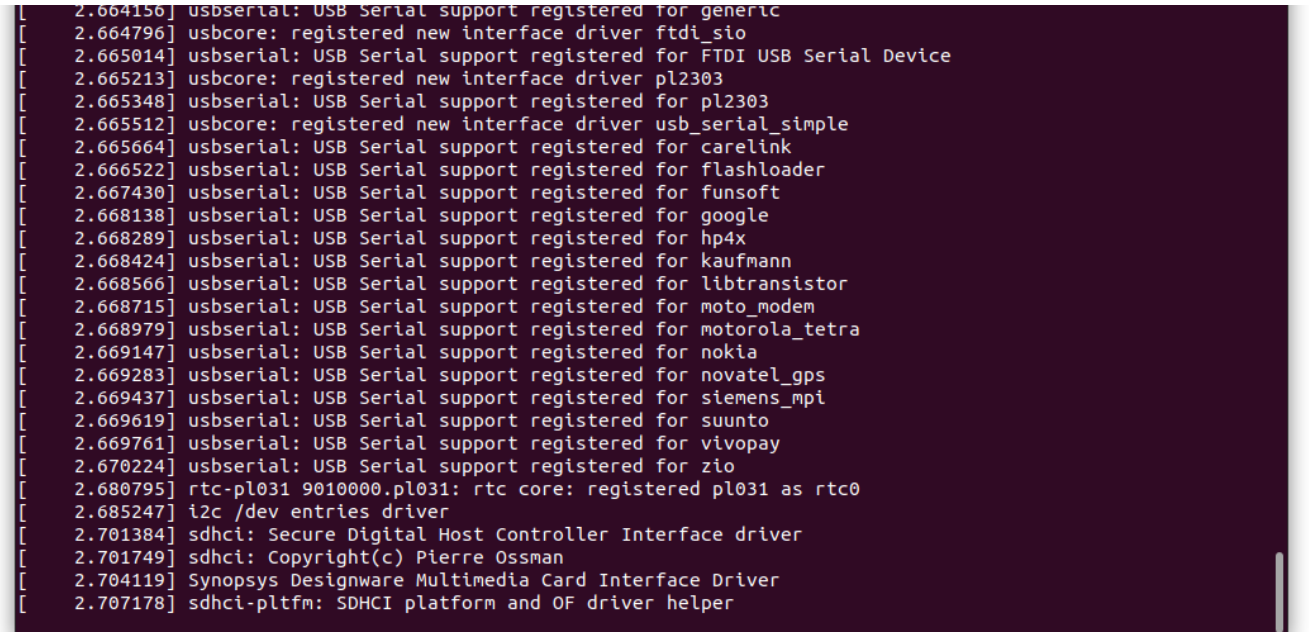


Рисунок 32 – Початок завантаження ядра в QEMU

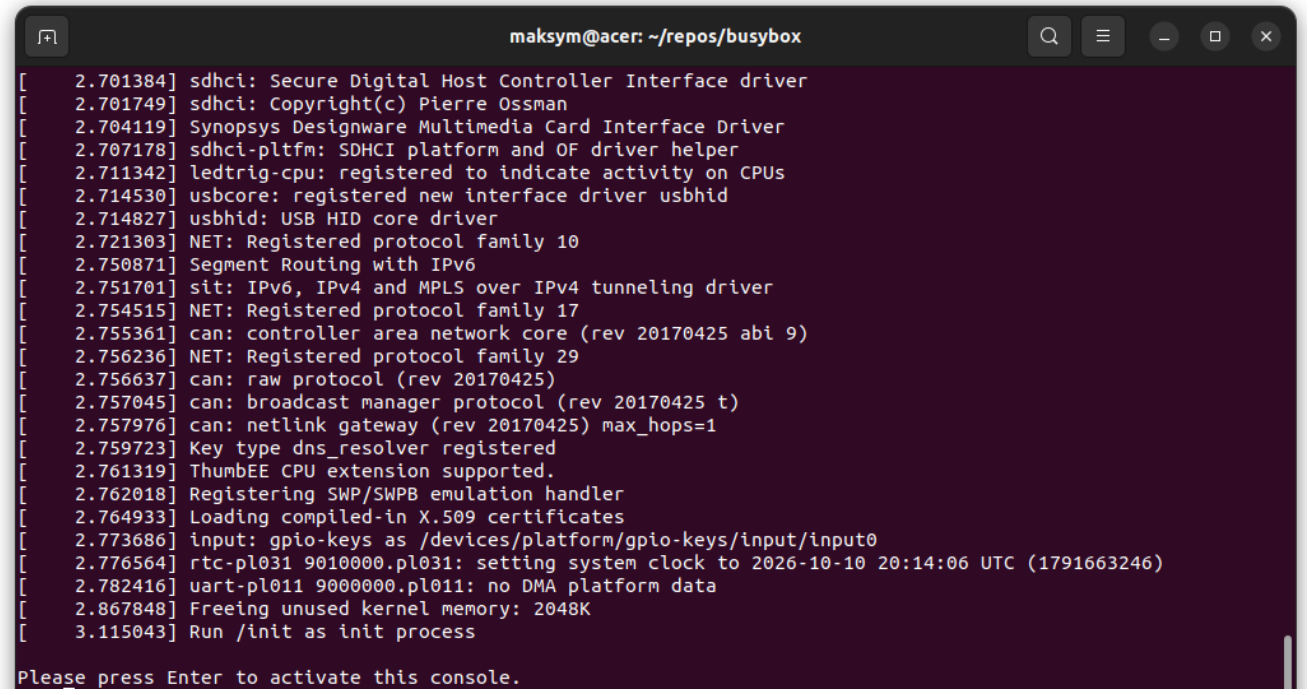


Рисунок 33 – Завершення завантаження системи

Після завершення завантаження натиснемо Enter і виконаємо команди, що підтверджують роботу зібраної системи.

```
uname -a
ls -l
dmesg | grep init
busybox --help | head -15
poweroff
```

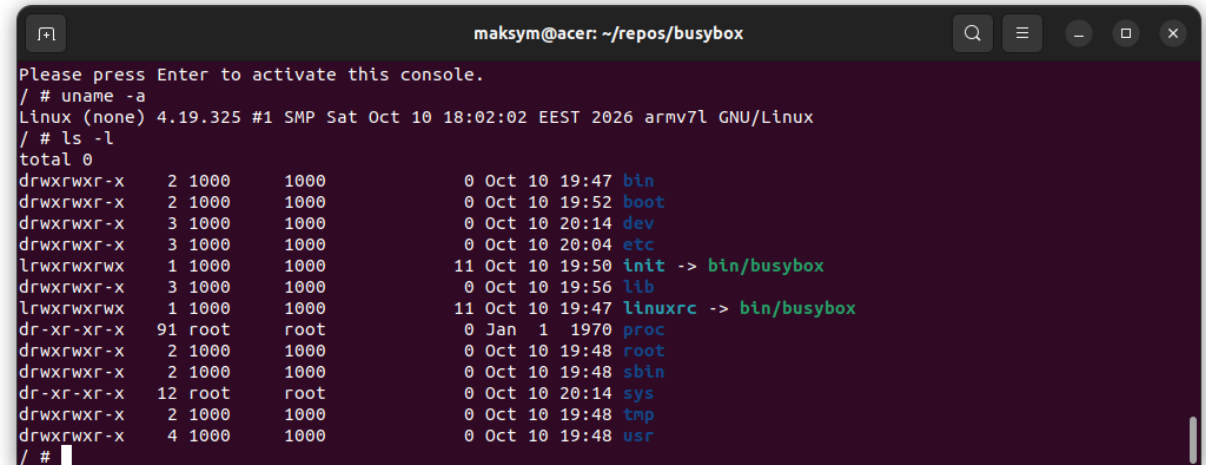


Рисунок 34 – Результат виконання uname -a та ls -l

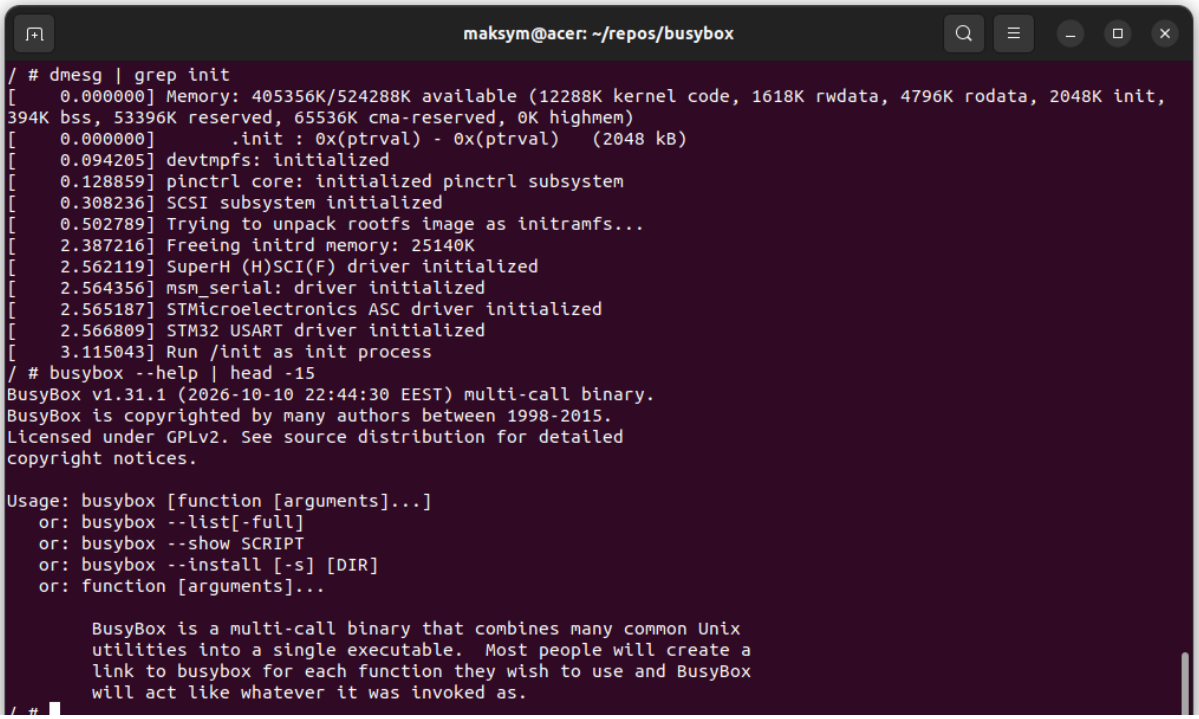


Рисунок 35 – Результат виконання dmesg та busybox --help

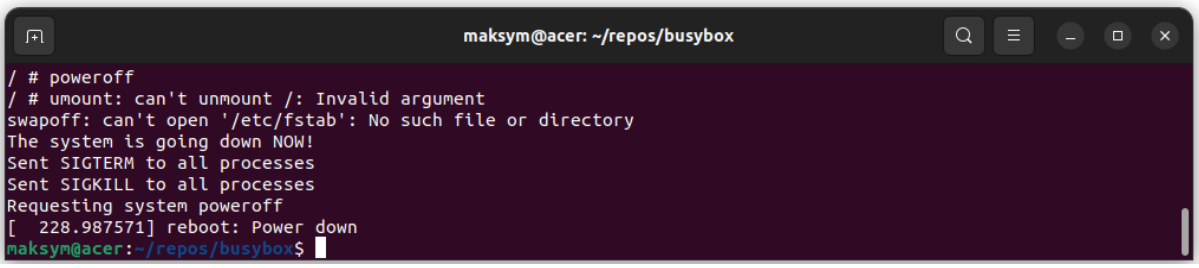


Рисунок 36 – Завершення роботи QEMU

Висновки:

Отже, під час виконання лабораторної роботи розгорнуто середовище для крос-компіляції під архітектуру ARM: встановлено ccache та два тулчейни (bare-metal і Linux). Зібрано завантажувач U-Boot (MLO та u-boot.img), ядро Linux 4.19 з конфігурацією multi_v7_defconfig і фрагментом для BeagleBone Black, а також мінімальну кореневу файлову систему на основі BusyBox із каталогами /boot, /lib та /etc. Отриману систему успішно запущено в емуляторі QEMU на машині virt, а виконані команди uname, ls, dmesg і busybox підтвердили її коректну роботу. Щоб уникнути помилки multiple definition of yylloc, яка виникає під час збірки U-Boot компілятором gcc 10 і новіших, ще до початку збірки було встановлено gcc 9 та перемкнено на нього систему, тому проблем під час виконання не виникло.